

Transitions: DS + Algorithms

Sorting



Today: An application of big-O + "pseudo code"

Let's start with a simple algorithm!

ALGORITHM 2 The Linear Search Algorithm.

procedure *linear search*(x : integer, $a_1, a_2, \dots, a_n$: distinct integers)

$i := 1$

while ($i \leq n$ and $x \neq a_i$)

$i := i + 1$

if $i \leq n$ **then** $location := i$

else $location := 0$

return $location$ { $location$ is the subscript of the term that equals x , or is 0 if x is not found}

Pseudocode format:

Varies by book! So we'll see a few.

In a pinch, pretend you're in Python or Ruby.

↳ high level & readable.

I realize this is not a "definition" -
that is the point!

It's about effective communication.

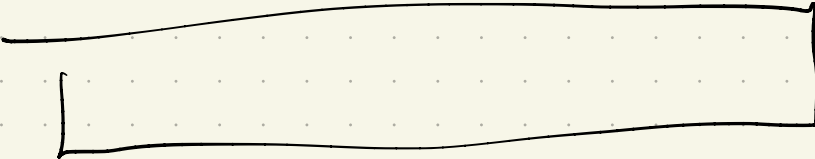
So: examples & practice!

Insertion sort:

Translate
to
pseudocode!

```
for (i = 1; i < n; i++) {  
    j = i;  
    while ((j > 0) && (s[j] < s[j - 1])) {  
        swap(&s[j], &s[j - 1]);  
        j = j - 1;  
    }  
}
```

Let's try

A: 

Runtime?

Bubble sort

```
procedure bubbleSort(A : list of sortable items)
  n := length(A)
  for i := 0 to n - 1 do
    for j := 0 to n - i - 2 do
      if A[j] > A[j + 1] then
        // Swap the elements
        temp := A[j]
        A[j] := A[j + 1]
        A[j + 1] := temp
      end if
    end for
  end for
end procedure
```

Demo:

Runtime:

Next reading: Recursion

Consider multiplication: is it really $O(1)$?

Input: 2 numbers
↳ in decimal

$X[0..m-1], Y[0..n-1]$

$$* \quad X = \sum_{i=0}^{m-1} X[i] \cdot 10^i, \quad Y = \sum_{j=0}^{n-1} Y[j] \cdot 10^j$$

Ex: 2 5 9 6 8

and

1 3 6 4

Back to grade school:

$$\begin{array}{r} 25968 \\ \times 1364 \\ \hline \end{array}$$

Essentially, for each $i \neq j$,
need $X[i] \cdot Y[j] \cdot 10^{c_i} \cdot 10^{c_j}$

$$\Rightarrow X \cdot Y = \sum_{i=1}^n \sum_{j=1}^m$$

Another view: Instead of just adding all,
search for all that land in one
spot k :

FIBONACCI MULTIPLY($X[0..m-1], Y[0..n-1]$):

$hold \leftarrow 0$

 for $k \leftarrow 0$ to $n + m - 1$

 for all i and j such that $i + j = k$

$hold \leftarrow hold + X[i] \cdot Y[j]$

$Z[k] \leftarrow hold \bmod 10$

$hold \leftarrow \lfloor hold / 10 \rfloor$

 return $Z[0..m + n - 1]$

What is k ?

Runtime?

Can we do better?

Claim 2

$$x \cdot y = \begin{cases} 0 & \text{if } x = 0 \\ \lfloor x/2 \rfloor \cdot (y + y) & \text{if } x \text{ is even} \\ \lfloor x/2 \rfloor \cdot (y + y) + y & \text{if } x \text{ is odd} \end{cases}$$

Why? Proof by cases:

if x is even:

if x is odd:

How to make an algorithm?

PEASANTMULTIPLY(x, y):

if $x = 0$

return 0

else

$x' \leftarrow \lfloor x/2 \rfloor$

$y' \leftarrow y + y$

$prod \leftarrow \text{PEASANTMULTIPLY}(x', y')$ *⟨⟨Recurse!⟩⟩*

if x is odd

$prod \leftarrow prod + y$

return $prod$